

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5

8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I G Vyshnavi (192525260) ____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:vyshnavi

Question 1 – Online Shopping Platform

Given Relation: Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Task: Identify all Functional Dependencies (FDs) and normalize the relation up to Third Normal Form (3NF).

Step 1: Understanding the Business Scenario

An online shopping platform records customer orders. A single order (identified by OrderID) can contain multiple products. Each product has a fixed price and name. Customer details are associated with the customer ID. Quantity is specific to a product within an order. This leads to redundancy if stored in a single flat table.

Step 2: Identification of Functional Dependencies

Based on real-world semantics, the following FDs hold:

- **CustID** → **CustName** (Customer name is uniquely determined by Customer ID)
- **ProdID** → **ProdName, Price** (Product name and price are attributes of the product)
- **OrderID** → **CustID** (Each order belongs to exactly one customer)
- **OrderID, ProdID** → **Qty** (Quantity of a product is specific to that order)

Step 3: Candidate Key

The minimal set of attributes that uniquely identifies a tuple is **{OrderID, ProdID}**. This is the candidate key (and primary key) of the original relation.

Step 4: Checking Normal Forms

1NF: The relation is already in 1NF as all attributes are atomic.

2NF Violation: There exist partial dependencies. CustName depends only on CustID (part of the key via OrderID), and ProdName/Price depend only on ProdID.

3NF: After removing partial dependencies we must also ensure no transitive dependencies remain.

Step 5: Decomposition into 3NF

The relation is decomposed into the following 3NF relations:

1. **Customer (CustID, CustName)** – Primary Key: CustID
2. **Product (ProdID, ProdName, Price)** – Primary Key: ProdID
3. **OrderHeader (OrderID, CustID)** – Primary Key: OrderID, Foreign Key: CustID references Customer
4. **OrderItem (OrderID, ProdID, Qty)** – Primary Key: (OrderID, ProdID), Foreign Keys reference OrderHeader and Product

Step 6: Justification

All relations are in 3NF because every non-prime attribute is fully functionally dependent on the primary key and there are no transitive dependencies. The decomposition is lossless (natural join

recovers the original relation) and dependency-preserving.

Question 2 – Hospital Patient Management

Given Relation: Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Task: Identify anomalies and normalize the relation up to Boyce-Codd Normal Form (BCNF).

Step 1: Identification of Functional Dependencies

- **PID** → **PName, DocID, WardNo**
- **DocID** → **DocName, Specialization**
- **WardNo** → **WardName**

Step 2: Anomalies Present

Insertion Anomaly: A new doctor who has not yet been assigned any patient cannot be inserted into the database because PID (primary key) would be null.

Deletion Anomaly: If the last patient of a particular doctor is deleted, the doctor's name and specialization information is also lost.

Update Anomaly: If a doctor's specialization or a ward's name changes, the change must be propagated to multiple rows, risking inconsistency.

Step 3: Candidate Key and Normal Form Analysis

Candidate Key of the original relation is {PID}. The relation is in 2NF but not in 3NF or BCNF because DocID and WardNo are determinants that are not candidate keys (transitive dependencies exist).

Step 4: BCNF Decomposition

1. **Patient (PID, PName, DocID, WardNo)** – Primary Key: PID
2. **Doctor (DocID, DocName, Specialization)** – Primary Key: DocID
3. **Ward (WardNo, WardName)** – Primary Key: WardNo

Step 5: Justification for BCNF

In each of the resulting relations, every determinant is a candidate key. Hence the design satisfies BCNF. The decomposition eliminates all anomalies while preserving data integrity through foreign-key relationships.

Question 3– University Enrollment System

Given Relation: Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

Task: Analyze Functional Dependencies and apply Second Normal Form (2NF) decomposition.

Step 1: Functional Dependencies

- **SID $\rightarrow$ SName**
- **CourseID $\rightarrow$ CourseName, Faculty**
- **SID, CourseID $\rightarrow$ Grade**

Step 2: Candidate Key

The composite candidate key is {SID, CourseID}. Grade is fully dependent on the entire key, but SName depends only on SID and CourseName/Faculty depend only on CourseID. These are partial dependencies.

Step 3: 2NF Decomposition

To achieve 2NF we remove partial dependencies by creating separate relations:

1. **Student (SID, SName)** – Primary Key: SID
2. **Course (CourseID, CourseName, Faculty)** – Primary Key: CourseID
3. **Enrollment (SID, CourseID, Grade)** – Primary Key: (SID, CourseID)

Step 4: Explanation

After decomposition, every non-prime attribute is fully functionally dependent on the primary key of its relation. Therefore the schema is in 2NF. Further normalization to 3NF is not required by the question, but the current design already has no transitive dependencies.

Question 4 – Airline Booking System

Given Relation: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Task: Normalize the relation up to Boyce-Codd Normal Form (BCNF).

Step 1: Functional Dependencies

- **PassID $\rightarrow$ PassName**
- **FlightNo $\rightarrow$ DeptCity, ArrCity**
- **FlightNo, PassID, Date $\rightarrow$ Seat**

Step 2: Candidate Key

The candidate key is the composite set {FlightNo, PassID, Date}. Seat is determined by the full key, while passenger name and flight cities are determined by proper subsets.

Step 3: BCNF Decomposition

1. **Passenger (PassID, PassName)** – Primary Key: PassID
2. **Flight (FlightNo, DeptCity, ArrCity)** – Primary Key: FlightNo
3. **Booking (FlightNo, PassID, Date, Seat)** – Primary Key: (FlightNo, PassID, Date)

Step 4: Justification

Each resulting relation has determinants that are candidate keys. Hence the schema is in BCNF. Redundancy of passenger names and city pairs is eliminated, and anomalies related to flight or passenger updates are removed.

Question 5 – HR System of a Multinational Company

Task: Identify all functional and transitive dependencies and normalize to 3NF. (A realistic HR schema is assumed below.)

Assumed Original Schema

Emp(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, Location, Salary)

Functional & Transitive Dependencies

- **EmpID** → EmpName, DeptID, ManagerID, Salary
- **DeptID** → DeptName, Location
- **ManagerID** → ManagerName
- **Transitive:** EmpID → DeptID → DeptName, Location and EmpID → ManagerID → ManagerName

3NF Decomposition

1. **Employee (EmpID, EmpName, DeptID, ManagerID, Salary)** – Primary Key: EmpID
2. **Department (DeptID, DeptName, Location)** – Primary Key: DeptID
3. **Manager (ManagerID, ManagerName)** – Primary Key: ManagerID

Justification

All transitive dependencies have been removed. Each non-prime attribute is non-transitively dependent on the primary key of its relation. The design is therefore in 3NF and free from update, insertion and deletion anomalies related to department or manager information.

Question 6 – Library Management System

Given Relation: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Task: Identify Multivalued Dependencies (MVDs) and decompose to Fourth Normal Form (4NF).

Step 1: Multivalued Dependencies

In a library context a member may borrow many books, a book may have multiple authors, and genre is an attribute of the book. This produces independent multi-valued facts:

- **MemberID** $\twoheadrightarrow$ **BookID** (a member can borrow many books independently)
- **BookID** $\twoheadrightarrow$ **Author** (a book can have several authors)
- **BookID** $\rightarrow$ **Title, Genre** (functional dependency)

Step 2: 4NF Decomposition

1. **Member (MemberID, MemberName)** – Primary Key: MemberID
2. **Book (BookID, Title, Genre)** – Primary Key: BookID
3. **BookAuthor (BookID, Author)** – Primary Key: (BookID, Author)
4. **Borrow (MemberID, BookID, BorrowDate)** – Primary Key: (MemberID, BookID, BorrowDate)

Step 3: Justification

The original relation violated 4NF because independent multivalued dependencies existed. After decomposition, each relation contains at most one multivalued dependency and the determinant is a candidate key, satisfying 4NF. Redundancy of member names, book titles and author lists is eliminated.

Question 7 – Telecom Billing Schema

Task: Identify all candidate keys, primary keys and functional dependencies before normalization. (A realistic telecom billing schema is assumed.)

Assumed Schema

Bill(CustID, CustName, PlanID, PlanName, CallID, CallDuration, CallCharge, BillDate, TotalAmount)

Functional Dependencies

- **CustID** $\rightarrow$ **CustName, PlanID**
- **PlanID** $\rightarrow$ **PlanName**
- **CallID** $\rightarrow$ **CustID, CallDuration, CallCharge, BillDate**
- **CustID, BillDate** $\rightarrow$ **TotalAmount**

Candidate Keys

Possible candidate keys: {CallID} (assuming CallID is globally unique) or the composite {CustID, CallID, BillDate}.

Primary Key chosen: CallID (simplest and unique identifier for each call record).

Observations before Normalization

The schema contains transitive dependencies ($\text{CustID} \rightarrow \text{PlanID} \rightarrow \text{PlanName}$) and partial dependencies if a composite key is used. These must be removed in later normalization steps to achieve 3NF/BCNF.

Question 8 – Retail Inventory System

Given Relation: Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Task: Apply complete normalization (up to BCNF).

Functional Dependencies

- $\text{PID} \rightarrow \text{PName, Category, Price, SupplierID, Warehouse}$
- $\text{SupplierID} \rightarrow \text{SupplierName}$

Analysis

The relation is in 2NF (single-attribute primary key) but violates 3NF because of the transitive dependency $\text{PID} \rightarrow \text{SupplierID} \rightarrow \text{SupplierName}$. Consequently it also violates BCNF.

Complete Normalization (BCNF)

1. **Product (PID, PName, Category, Price, SupplierID, Warehouse)** – Primary Key: PID
2. **Supplier (SupplierID, SupplierName)** – Primary Key: SupplierID

Justification

Both relations satisfy BCNF: the only determinants are the respective primary keys. Supplier name is stored only once, eliminating update anomalies. Foreign-key integrity can be maintained between Product.SupplierID and Supplier.SupplierID.

Question 9 – School ERP System

Task: Study a poorly normalized schema and propose a fully normalized design up to BCNF with justification.

Assumed Poorly Normalized Schema

StudentRecord(SID, SName, Class, Subject, Teacher, Marks, ParentPhone)

Problems in the Original Design

- Partial dependency: SName and ParentPhone depend only on SID.
- Transitive dependency: Subject determines Teacher in many school systems.
- Redundancy: Class and ParentPhone are repeated for every subject a student takes.

Proposed BCNF Design

1. **Student (SID, SName, Class, ParentPhone)** – Primary Key: SID
2. **Subject (SubjectID, SubjectName)** – Primary Key: SubjectID
3. **Teacher (TeacherID, TeacherName)** – Primary Key: TeacherID
4. **ClassSubject (Class, SubjectID, TeacherID)** – Primary Key: (Class, SubjectID)
5. **Marks (SID, SubjectID, Marks)** – Primary Key: (SID, SubjectID)

Justification

Every determinant in the new schema is a candidate key, satisfying BCNF. All partial and transitive dependencies have been removed. The design supports referential integrity through foreign keys and eliminates redundancy of student personal data and teacher assignments.

Question 10 – Movie Streaming Platform

Task: Identify join dependencies and decompose to Fifth Normal Form (5NF) with lossless-join verification.

Assumed Schema

Stream(UserID, MovieID, Actor, Genre, WatchDate)

Join Dependency

The relation satisfies the join dependency: $\star(\text{UserID}, \text{MovieID}, \text{WatchDate}) \bowtie (\text{MovieID}, \text{Actor}) \bowtie (\text{MovieID}, \text{Genre})$. Actors and genres are independent properties of a movie, while watch history is independent of both.

5NF Decomposition

1. **WatchHistory (UserID, MovieID, WatchDate)** – Primary Key: (UserID, MovieID, WatchDate)

2. **MovieActor (MovieID, Actor)** – Primary Key: (MovieID, Actor)
3. **MovieGenre (MovieID, Genre)** – Primary Key: (MovieID, Genre)

Lossless-Join Verification

The natural join of the three projections recovers exactly the original relation without generating spurious tuples. This can be verified by the fact that the common attributes (MovieID) form a key in the actor and genre relations, guaranteeing lossless decomposition.

Justification

The schema is now in 5NF because it cannot be further non-loss decomposed. All join dependencies have been isolated into separate relations, removing the redundancy that arises when actors and genres are repeated for every user who watches a movie.